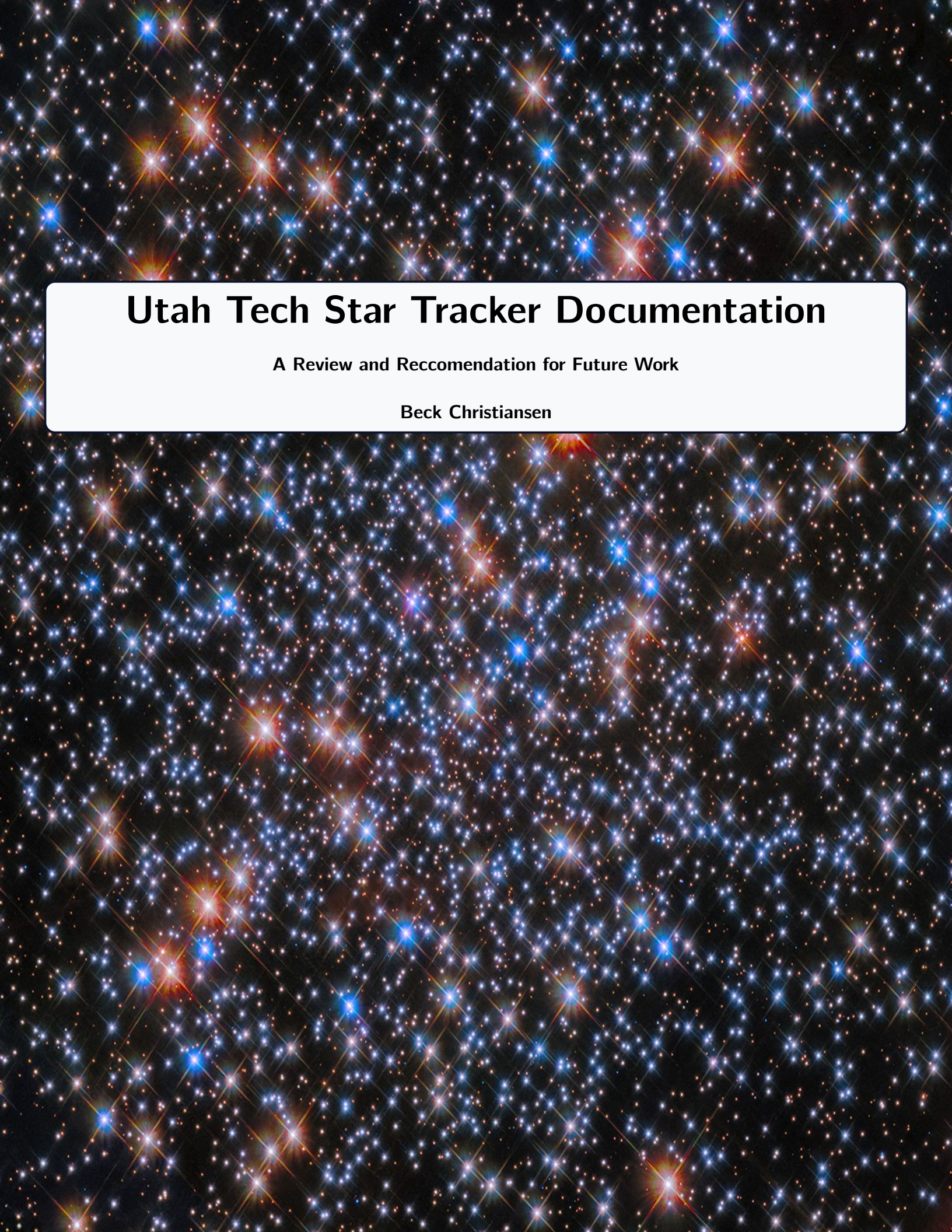

Utah Tech Star Tracker Documentation

A Review and Reccomendation for Future Work

Beck Christiansen

1 Introduction

Hello and welcome to the Utah Tech CubeSat community! This is a complex, multifacited project that can be simultaniously rewarding and challenging. The goal of this paper is twofold. First, I aim to familiarize you with the inner workings of a star tracker. Hopefully, after reading this you will be familiar with the language and notation assosiated with star trackers. You will understand at a high level how a star tracker (specifically our star tracker) works. you will understand the mathmatics and logic needed to make it run. You will understand how to use the custom build UTU StarTracker class to determine a satellite's orientation.

The second goal of mine is to give you some guidance on how to move forward with the project. You will understand the limitations of our system, why certain design decisions were made, and what needs to be changed. At the end of this paper a comprehensive list of known issues, and solutions will be presented.

2 Background

Before we dive into technical discussion, we need to discuss what a star tracker is and what is does. Lets start with a thought experiment. Imagine for a moment you are magically teleported to a random spot in earth's orbit. You don't know where you are or what direction you are pointed. How do you figure out your orientation?

To solve this thought experiment I need to introduce coordinate frames. For a formal definition review a mechanics textbook, but informally, a coordinate frame is a set of orthogonal axes and an origin used to define position, orientation, velocity, etc. An inertial coordinate frame is one that does not accelerate/rotate.

Ok now lets go back to the thought experiment. You need to find out where you are pointed. Your first thought might be to look at the earth. We could see what part of earth we are flying over, convert those coordinates from the rotating frame (ECEF) to an inertial frame (ECI) and and then we know our attitude. There are two potential problems. First, what if we are not pointed at the earth? Second, what if clouds are covering the surface, or what if we are over the dark side of the earth?

Your next thought might be to use the location of the sun to determine your attitude. This is actually a fairly common method which uses orbital mechanics and sun sensors to determine your orientation. Depending on your mission requirements, the fidelity might not be high enough. Sun sensors can have accuracies as bad as $\pm 10°$. For fine pointing that is not sufficient.

This brings us to the advent of the star tracker. Lets make a few assumptions and definitions. First we are going to assume that the origin of our coordinate frame is the center of earth with the z-axis along the north pole, and the x-axis pointing towards the vernal equinox. Second we are going to assume that the coordinate frame is inertial, meaning the earth stays "fixed" in space and the sun "orbits" the earth. This also means that the frame does not rotate with earth. The name of this coordinate frame is the Earth Centered Inertial (ECI) coordinate frame. A star tracker determines orientation by taking a photo of the stars, comparing that image to a star map, and determining the ECI coordinates based off the matched stars. This is a very robust method for attitude determination because it works during any point of an orbit, it is very accurate, and it is relatively fast.

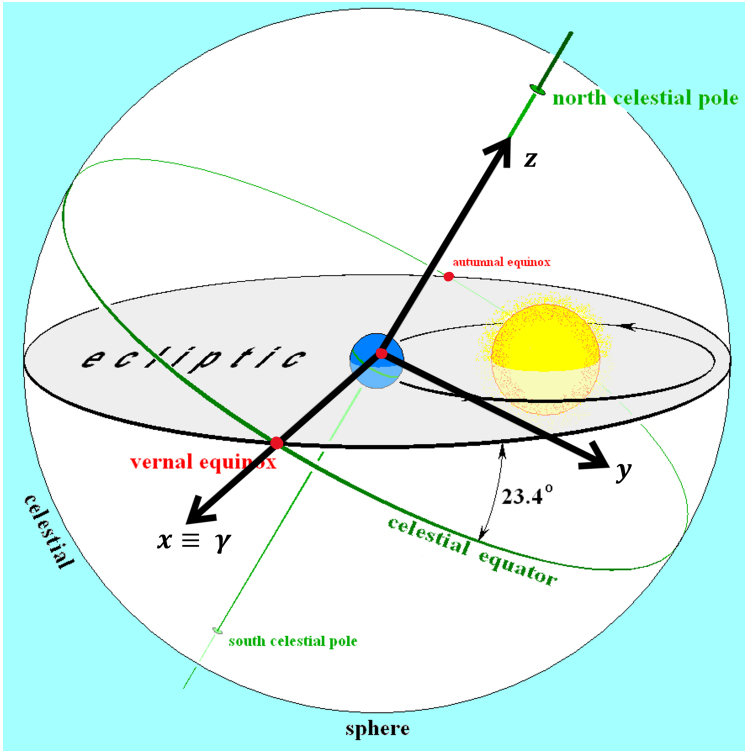

Figure 1: An image illustrating the ECI coordinate frame.

3 Installing the Star Tracker

I will briefly go over how to install the star tracker and get it up and running. In order to run the software you will need a computer with some sort of Linux distribution installed. For night sky testing I recommend using a raspberry pi 4 with the raspberry pi camera v2.1 (we should have both in our materials cabinet). If you plan on further developing the code, I highly recommend installing the software on a laptop/desktop with Linux. working exclusively on the raspberry pi gets annoying fast.

To download the software follow the standard GitHub workflow either cloning the repository or downloading the zip file. once the repo is setup navigate to the `RPi` directory. Once in the directory, you will see two shell scripts: `extract_cat.sh` and `linux_installer.sh`. These programs install the star catalog and astronomy programs respectively. to run the programs enter the following commands into the terminal:

- `sh linux_installer.sh`
- `sh extract_cat.sh`

Once both scripts have been run, the software is good to go.

4 System Overview

In this section I aim to give a high level overview of the star tracker system, specifically the star tracker class written for the UTU CubeSat program. While a low level explanation of every method may prove useful for future students I simply do not have the time or bandwidth to complete such a task. For detailed information I recommend reading through the source code as well as checking out the SPEL Open Star Tracker (SOST) project linked here. The UTU Star Tracker is derived from this project and therefore is HEAVILY related. The following sub-sections will detail the properties and methods associated with the star tracker class and how to use them.

4.1 take_photo()

This method uses a camera attached to the raspberry pi to snap a photo of the stars. It uses a command line tool called rpicam to run. In order to use this method a library called subprocess must be used. This library allows python to interact with the computer's terminal. This method needs no input parameters aside from the class properties.

4.2 pull_photo()

This method is used as an alternative to `take_photo()`. It loads a specified photo from a designated archive and treats it as a photo the camera just took. This is useful for testing the star tracker during daytime. The archive `utu_img` is a folder of 640 sky images taken in Ivins Utah. It has been used to test the star tracker system.

4.3 jpg2fits()

This method takes a jpeg photo and converts it to a fits file. A fits file is an image commonly used in astronomy. This conversion is necessary because a tool used later (source-extractor) needs images in that format.

4.4 apply_sextractor()

This method uses the fits image generated in the last method, and feeds it into a program called source extractor. source extractor finds all the points of light in an image (in this case the stars) and assigns them coordinates. The output of this method is a list of all the coordinates of the stars in the captured image. for a more in depth explination of source extractor, read its documentation linked here.

4.5 map_match_and_radec_list_multiprocess()

To speed up the identification process, this method utilizes the Raspberry Pi's multiple CPU cores. It simultaneously compares the stars found in your photo against several different segments of the star catalog. This "parallel processing" significantly reduces the time it takes to find a potential match.

4.6 call_match()

This is a static helper method that acts as a bridge between Python and the "Match" C-program. It formats the specific command-line strings required to run the matching algorithm and returns whether the search was successful. The Match documentation is linked here.

4.7 get_table_with_matchs()

After the multi-processed search is finished, this method gathers all the successful results into a single table. It sorts these candidates based on the number of stars matched and the "offset error," ensuring that the most likely sky location is at the top of the list.

4.8 get_match_candidates()

This method acts as a filter that selects the top few candidates from the match table to be passed into the refinement stage. This ensures the system doesn't waste time processing low-probability matches.

4.9 apply_match_trans()

Once a match is found, the coordinates are still in a raw format. This method applies a mathematical transformation to convert those results into meaningful physical values, specifically the right ascension (RA), declination (DEC), and the roll angle of the camera.

4.10 plane2sky()

This method performs a "deprojection." It takes coordinates from the flat 2D plane of the camera sensor and maps them back onto the 3D "celestial sphere." This is the step that translates "pixels" or "millimeters on a sensor" into actual coordinates in space.

4.11 sky2plane()

The inverse of the previous method, this projects 3D star catalog coordinates onto a 2D flat surface. This is used during the refinement process to create a "predicted" image of what the camera should be seeing, which can then be compared to the actual photo.

4.12 search_catalog_objects()

This method looks through the local star database to find known stars near a specific set of coordinates. It helps the system "zoom in" on a specific area of the sky once an initial guess has been made.

4.13 match_itr()

This is the core "logic loop" of the solver. It iteratively refines the star tracker's guess. It starts with a rough estimate of where the camera is pointing and repeats the matching and projection process several times to "lock on" to the most accurate orientation possible.

4.14 ECI_2_QUAT()

This is the final mathematical step. It takes the Right Ascension, Declination, and Roll and converts them into a Quaternion. A quaternion is a four-number vector used in aerospace to represent 3D orientation without the mathematical risks of "gimbal lock."

4.15 solve_LIS()

This is the "Master Method" that orchestrates the entire system. It calls the previous methods in the correct order—from taking the photo to outputting the final quaternion. "LIS" stands for Lost-In-Space, referring to the condition where the satellite has no prior knowledge of its orientation and must figure it out from scratch.

5 Future Work

There are a few things that can be done to improve the performance of the star tracker. First is adding an algorithm/method that moves beyond the Lost-In-Space problem. once we have solved the LIS problem once we have a good idea of where we are during later time steps. writing a dynamic program that uses the previous orientation as our initial search in the star catalog will bring down the computation drastically.

Second, to bring down the error of the system a better camera could be used. A camera, such as the Raspberry Pi HQ Camera M12, would give the system a much higher fidelity. With the better image quality and better field of view

the system could achieve much better standard deviations in pointing accuracy.

Finally, when integrating the star tracker into the larger CubeSat system, a greater emphasis must be placed on error handling. The current setup is great for proof of concept, but in orbit the star tracker must work all of the time. if an unexpected error occurs, the code must catch it. Additionally, it may be advantageous to design the system to allow for adaptive parameter tuning. The system can only be tested on earth, but what if the parameters that work well on earth are not as good in space? Designing an adaptive system would allow for parameters to be adjusted "on the fly" which could potentially save a mission.